	Final: Part A (20 Marks)		
4.	**Tools of analysis and design:** Data Flow Diagram (DFD), DFD symbols, Constructing DFD; Data Dictionary; Decision Tree, Structured English, Decision Tables.	02 weeks	CLO4
5.	**System Design and Construction:** The process of design, System design phases, Design methodologies; Structured design; Form-Driven methodology; Input design, Output design, File and database design	02 weeks	CLO4
	Final: Part B (30 Marks)		
6.	**Testing and Quality Assurance:** Testing, Types of system tests; White-Box testing; Black-box testing; Quality factors specifications.	01 week	CLO2
7.	**Implementation and Maintenance:** Types of implementations, Documenting the system, Training and supporting users, Factor models of implementation success; The process of maintaining information system, Types of maintenance, Cost of maintenance, Reducing maintenance cost.	02 weeks	CLO2
8.	**Hardware/ Software selection, control and security:** Phases in selection, Criteria for software selection, Hardware selection, Financial considerations in selection; Security definitions, Threats to system security, Control measures, system failures and recovery.	02 weeks	CLO2

Segment - 4

What is Structured Analysis?

Structured Analysis is a development method that allows the analyst to understand the system and its activities in a logical way.

It is a systematic approach, which uses graphical tools that analyze and refine the objectives of an existing system and develop a new system specification which can be easily understandable by user.

It has following attributes −

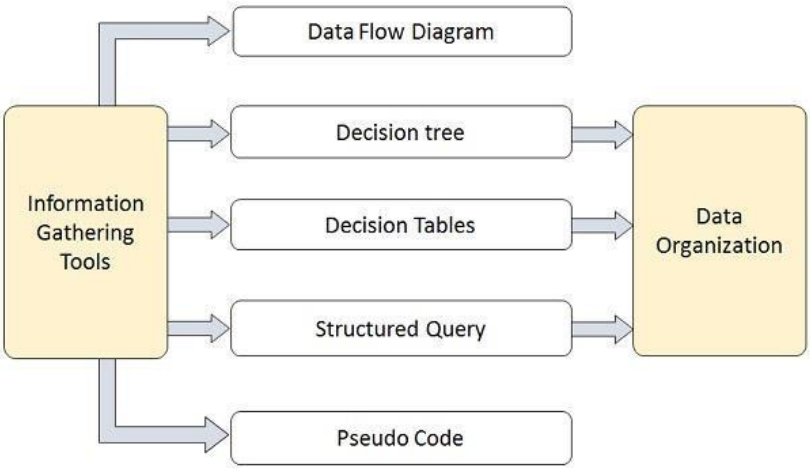
- It is graphic which specifies the presentation of application.
- It divides the processes so that it gives a clear picture of system flow.
- It is logical rather than physical i.e., the elements of system do not depend on vendor or hardware.
- It is an approach that works from high-level overviews to lower-level **details.**

Structured Analysis Tools

During Structured Analysis, various tools and techniques are used for system development.

They are −

1. Data Flow Diagrams
2. Data Dictionary
3. Decision Trees
4. Decision Tables
5. Structured English
6. Pseudocode



Data Flow Diagrams (DFD) or Bubble Chart

It is a technique developed by Larry Constantine to express the requirements of system in a graphical form.

- It shows the flow of data between various functions of system and specifies how the current system is implemented.
- It is an initial stage of design phase that functionally divides the requirement specifications down to the lowest level of detail.
- Its graphical nature makes it a good communication tool between user and analyst or analyst and system designer.
- It gives an overview of what data a system processes, what transformations are performed, what data are stored, what results are produced and where they flow.

Types of DFD

DFDs are of two types: Physical DFD and Logical DFD. The following table lists the points that differentiate a physical DFD from a logical DFD.

Physical DFD	Logical DFD
It is implementation dependent. It shows which functions are performed.	It is implementation independent. It focuses only on the flow of data between processes.
It provides low level details of hardware, software, files, and people.	It explains events of systems and data required by each event.
It depicts how the current system operates and how a system will be implemented.	It shows how business operates; not how the system can be implemented.

Design Feature	Logical	Physical
What the model depicts	How the business operates.	How the system will be implemented (or how the current system operates).
What the processes represent	Business activities.	Programs, program modules, and manual procedures.
What the data stores represent	Collections of data regardless of how the data are stored.	Physical files and databases, manual files.
Type of data stores	Show data stores representing permanent data collections.	Master files, transition files. Any processes that operate at two different times must be connected by a data store.
System controls	Show business controls.	Show controls for validating input data, for obtaining a record (record found status), for ensuring successful completion of a process, and for system security (example: journal records).

Data Dictionary

A data dictionary is a structured repository of data elements in the system. The data dictionary is a reference work of data about data (that is, metadata), one that is compiled by systems analysts to guide them through analysis and design. It stores the descriptions of all DFD data elements that is, details and definitions of data flows, data stores, data stored in data stores, and the processes.

One important reason for maintaining a data dictionary is to keep clean data. This means that data must be consistent. If you store data about a man's sex as "M" in one record, "Male" in a second record, and as the number "1" in a third record, the data are not clean. Keeping a data dictionary will help in this regard.

The data dictionary is created by examining and describing the contents of the data flows, data stores, and processes, as illustrated by Figure. Each data store and data flow should be defined and then expanded to include the details of the elements it contains. The logic of each process should be described using the data flowing into or out of the process.

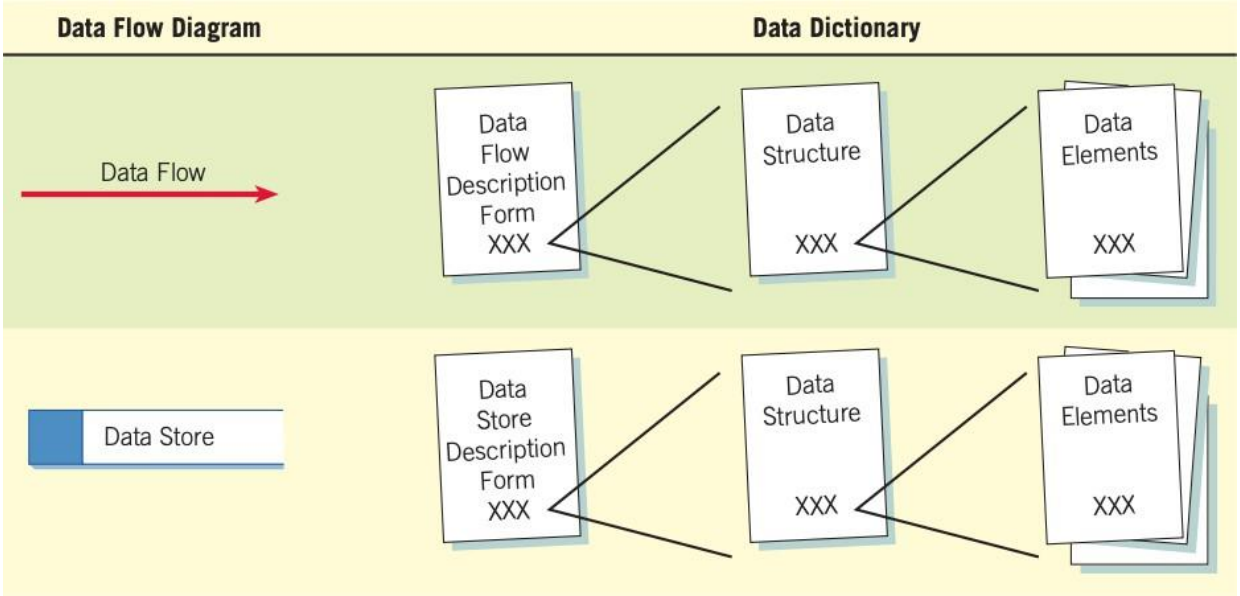


Figure: How data dictionaries relate to data flow diagrams.

A data dictionary improves the communication between the analyst and the user. It plays an important role in building a database. Most DBMSs have a data dictionary as a standard feature. For example, refer the following table −

Sr.No.	Data Name	Description	No. of Characters
1	ISBN	ISBN Number	10
2	TITLE	title	60
3	SUB	Book Subjects	80
4	ANAME	Author Name	15

Process Specification Format

Process specifications—sometimes called ***"minispecs",*** because they are a small portion of the total project specifications—are created for primitive processes on a data flow diagram as well as for some higher-level processes that explode to a child diagram. These specifications explain the decision-making logic and formulas that will transform process input data into output.

Process specifications link the process to the data flow diagram, and hence the data dictionary, as illustrated in Figure.

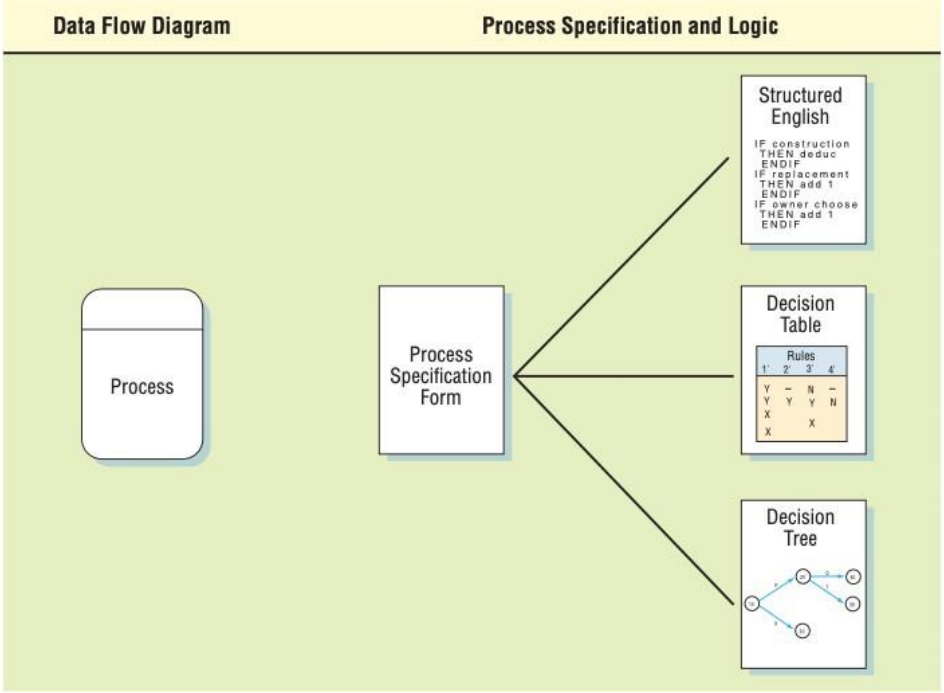


Figure: How process specifications relate to the data flow diagram.

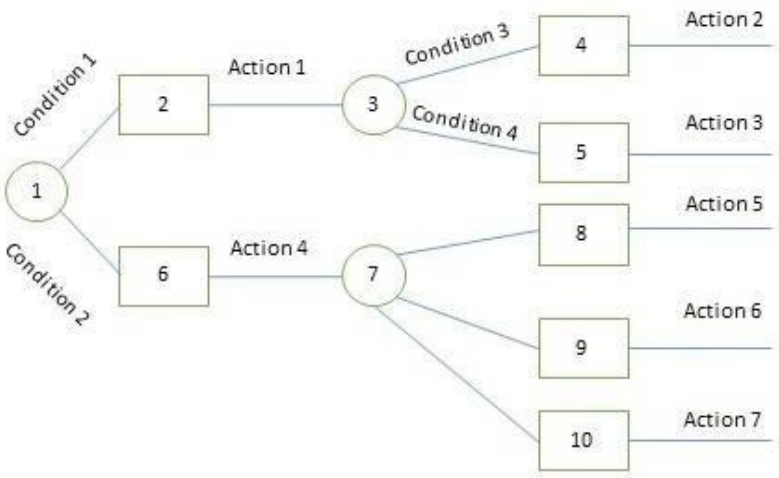
Decision Trees

Decision trees are used when complex branching occurs in a structured decision process. Trees are also useful when it is essential to keep a string of decisions in a particular sequence. Decision trees are a method for defining complex relationships by describing decisions and avoiding the problems in communication. A decision tree is a diagram that shows alternative actions and conditions within horizontal tree framework. Thus, it depicts which conditions to consider first, second, and so on.

Decision trees depict the relationship of each condition and their permissible actions. A square node indicates an action and a circle indicates a condition. It forces analysts to consider the sequence of decisions and identifies the actual decision that must be made.

In drawing the tree:

1. Identify all conditions and actions and their order and timing (if they are critical).
2. Begin building the tree from left to right, making sure you list all possible alternatives before moving to the right.



The major limitation of a decision tree is that it lacks information in its format to describe what other combinations of conditions you can take for testing. It is a single representation of the relationships between conditions and actions.

For example, refer the following decision tree −



Decision Tables

Decision tables are a method of describing the complex logical relationship in a precise manner which is easily understandable.

It is useful in situations where the resulting actions depend on the occurrence of one or several combinations of independent conditions.

It is a matrix containing row or columns for defining a problem and the actions.

Components of a Decision Table

Condition Stub − It is in the upper left quadrant which lists all the condition to be checked.

Action Stub − It is in the lower left quadrant which outlines all the action to be carried out to meet such condition.

Condition Entry − It is in upper right quadrant which provides answers to questions asked in condition stub quadrant.

Action Entry − It is in lower right quadrant which indicates the appropriate action resulting from the answers to the conditions in the condition entry quadrant.

Conditions and Actions	Rules
Conditions	Condition Alternatives
Actions	Action Entries

Figure: The standard format used for presenting a decision table.

The entries in decision table are given by Decision Rules which define the relationships between combinations of conditions and courses of action. In rules section,

- Y shows the existence of a condition.
- N represents the condition, which is not satisfied.
- A blank - against action states it is to be ignored.
- X (or a check mark will do) against action states it is to be carried out.

For example, refer the following table −

CONDITIONS	Rule 1	Rule 2	Rule 3	Rule 4
Advance payment made	Y	N	N	N
Purchase amount = Rs 10,000/-	-	Y	Y	N
Regular Customer	-	Y	N	-
ACTIONS				
Give 5% discount	X	X	-	-
Give no discount	-	-	X	X

Example-1

Example-2:

Suppose a store wanted to illustrate its policy on noncash customer purchases. The company could do so using a simple decision table as shown in Figure. Each of the three conditions (sale under $50, pays by check, and uses credit cards) has only two alternatives. The two alternatives are Y (yes, it is true) or N (no, it is not true). Four actions are possible:

1. Complete the sale after verifying the signature.
2. Complete the sale. No signature needed.
3. Call the supervisor for approval.
4. Communicate electronically with the bank for credit card authorization.

The final ingredient that makes the decision table worthwhile is the set of rules for each of the actions. Rules are the combinations of the condition alternatives that precipitate an action. For example, Rule 3 says:

IF	N	(the total sale is NOT under $50.00)
		AND
IF	Y	(the customer paid by check and had two forms of ID)
		AND
IF	N	(the customer did not use a credit card)
		THEN
DO	X	(call the supervisor for approval).

The foregoing example featured a problem with four sets of rules and four possible actions, but that is only a coincidence.

Conditions and Actions	Rules 1	2	3	4
Under $50	Y	Y	N	N
Pays by check with two forms of ID	Y	N	Y	N
Uses credit card	N	Y	N	Y
Complete the sale after verifying signature.	X			
Complete the sale. No signature needed.		X		
Call supervisor for approval.			X	
Communicate electronically with bank for credit card authorization.				X

Figure: Using a decision table for illustrating a store's policy of customer checkout with four sets of rules and four possible actions.

Structured English

Structured English is derived from structured programming language which gives more understandable and precise description of process. It is based on procedural logic that uses construction and imperative sentences designed to perform operation for action.

When the process logic involves formulas or iteration, or when structured decisions are not complex, an appropriate technique for analyzing the decision process is the use of structured English. As the name implies, structured English is based on (1) structured logic, or instructions organized into nested and grouped procedures, and (2) simple English statements such as add, multiply, and move. A word problem can be transformed into structured English by putting the decision rules into their proper sequence and using the convention of IF-THEN-ELSE statements throughout.

- It is best used when sequences and loops in a program must be considered and the problem needs sequences of actions with decisions.
- It does not have strict syntax rule. It expresses all logic in terms of sequential decision structures and iterations.

Structured English Type	Example
Sequential Structure A block of instructions in which no branching occurs	Action #1 Action #2 Action #3
Decision Structure Only IF a condition is true, complete the following statements; otherwise, jump to the ELSE	IF Condition A is True THEN implement Action A ELSE implement Action B ENDIF
Case Structure A special type of decision structure in which the cases are mutually exclusive (if one occurs, the others cannot)	IF Case #1 implement Action #1 ELSE IF Case #2 Implement Action #2 ELSE IF Case #3 Implement Action #3 ELSE IF Case #4 Implement Action #4 ELSE print error ENDIF
Iteration Blocks of statements that are repeated until done	DO WHILE there are customers. Action #1 ENDDO

Examples of logic expressed in a sequential structure, a decision structure, a case structure, and an iteration.

For example, see the following sequence of actions −

```
if customer pays advance
   then
      Give 5% Discount
   else
      if purchase amount >=10,000
         then
            if  the customer is a
               then Give 5% Discou
            else  No Discount
         end if
      else No Discount
   end if
end if
```

Pseudocode

A pseudocode does not conform to any programming language and expresses logic in plain English.

- It may specify the physical programming logic without actual coding during and after the physical design.
- It is used in conjunction with structured programming.
- It replaces the flowcharts of a program.

Guidelines for Selecting Appropriate Tools

Use the following guidelines for selecting the most appropriate tool that would suit your requirements −

- Use DFD at high or low level analysis for providing good system documentations.
- Use data dictionary to simplify the structure for meeting the data requirement of the system.
- Use structured English if there are many loops and actions are complex.
- Use decision tables when there are a large number of conditions to check and logic is complex.
- Use decision trees when sequencing of conditions is important and if there are few conditions to be tested.

The following guidelines provide you with a way to choose one of the three techniques for a particular case:

1. Use structured English when
   a. There are many repetitious actions,
      OR
   b. Communication to end users is important.
2. Use decision tables when
   a. Complex combinations of conditions, actions, and rules are found,
      OR
   b. You require a method that effectively avoids impossible situations, redundancies, and contradictions.
3. Use decision trees when
   a. The sequence of conditions and actions is critical,
      OR
   b. When not every condition is relevant to every action (the branches are different).

Segment 4: Tools of analysis and design

Process modeling:

- A process model describes business processes—the activities that people do. Process models are developed for the as-is system and/or the to-be system.
- Data and process modeling involves three main tools: data flow diagrams, a data dictionary, and process descriptions.

Data Flow Diagram (DFD):

- A data flow diagram (DFD) uses various symbols to show how the system transforms input data into useful information.
- A data flow diagram (DFD) shows how data moves through an information system but does not show program logic or processing steps.
- A set of DFDs provides a logical model that shows *what* the system does, not *how* it does it.

DFD Symbols:

- DFDs use four basic symbols that represent processes, data flows, data stores, and entities.
- There are two type of symbol set. One is ***Gane and Sarson*** symbol set. Another popular symbol set is the ***Yourdon*** symbol set.

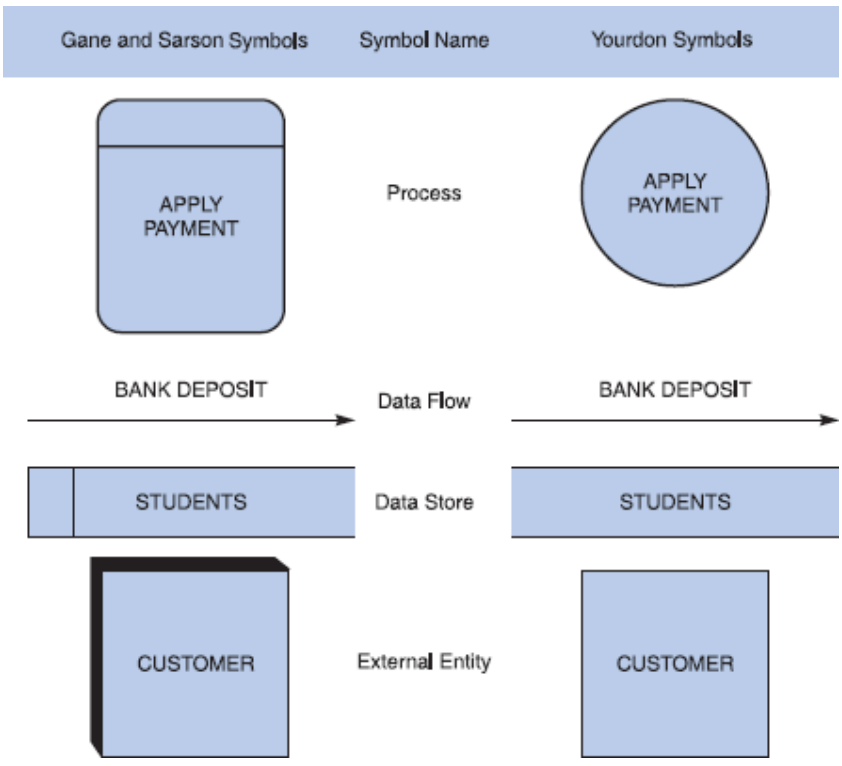


Figure 1: Data flow diagram symbols, symbol names, and examples of the Gane and Sarson and Yourdon symbol sets.

Process Symbol:

- A process receives input data and produces output that has a different content, form, or both. For instance, the process for calculating pay uses two inputs (pay rate and hours worked) to produce one output (total pay).
- Processes contain the ***business logic***, also called ***business rules*** that transform the data and produce the required results.
- The process name identifies a specific function and consists of a verb (and an adjective, if necessary) followed by a singular noun. Examples of process names are APPLY RENT PAYMENT, CALCULATE COMMISSION, ASSIGN FINAL GRADE, VERIFY ORDER, and FILL ORDER.
- Processing details are not shown in a DFD. For example, you might have a process named DEPOSIT PAYMENT. The process symbol does not reveal the business logic for the DEPOSIT PAYMENT process. To document the logic, you create a process description.

Data Flow Symbol:

- A data flow is a path for data to move from one part of the information system to another.
- A data flow name consists of a singular noun and an adjective, if needed. Examples of data flow names are DEPOSIT, INVOICE PAYMENT, STUDENT GRADE, ORDER, and COMMISSION.

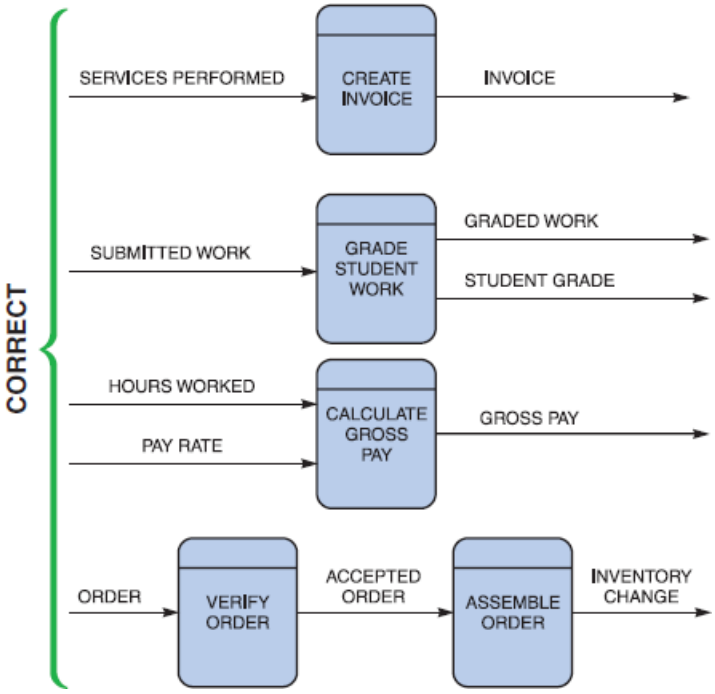


Figure 2: correct examples of data flow and process symbol connections.

Figure 3 shows three data flow and process combinations that you must avoid:

- *Spontaneous generation*: The APPLY INSURANCE PREMIUM process, for instance, produces output, but has no input data flow. Because it has no input, the process is called a spontaneous generation process.
- *Black hole:* The CALCULATE GROSS PAY is called a black hole process, which is a process that has input, but produces no output.
- *Gray hole:* A gray hole is a process that has at least one input and one output, but the input obviously is insufficient to generate the output shown. For example, a date of birth input is not sufficient to produce a final grade output in the CALCULATE GRADE process.

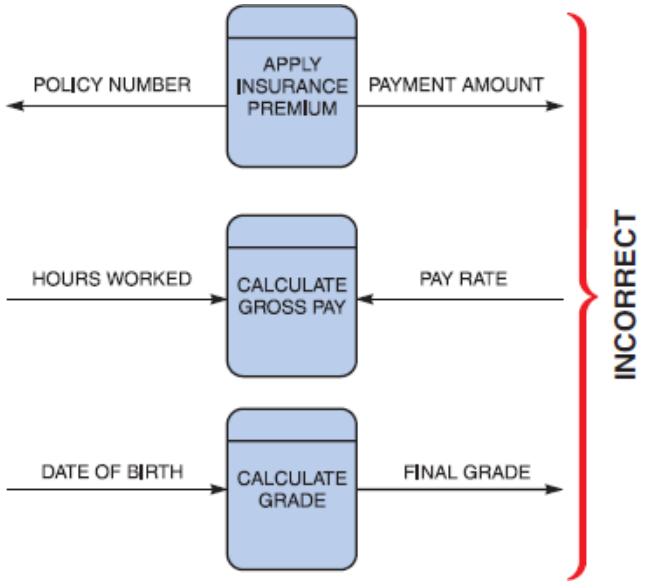


Figure 3: Examples of incorrect combinations of data flow and process symbols.

Data Store Symbol

- A data store is used in a DFD to represent data that the system stores because one or more processes need to use the data at a later time
- A DFD does not show the detailed contents of a data store — the specific structure and data elements are defined in the data dictionary.
- A data store name is a plural name consisting of a noun and adjectives, if needed. Examples of data store names are STUDENTS, ACCOUNTS RECEIVABLE, PRODUCTS, DAILY PAYMENTS.
- In each case, the data store has at least one incoming and one outgoing data flow and is connected to a process symbol with a data flow.
- In some situations, a data store has no input data flow because it contains fixed reference data that is not updated by the system. For example, consider a data store called TAX TABLE, which contains withholding tax data that a company downloads from the Internal Revenue Service.

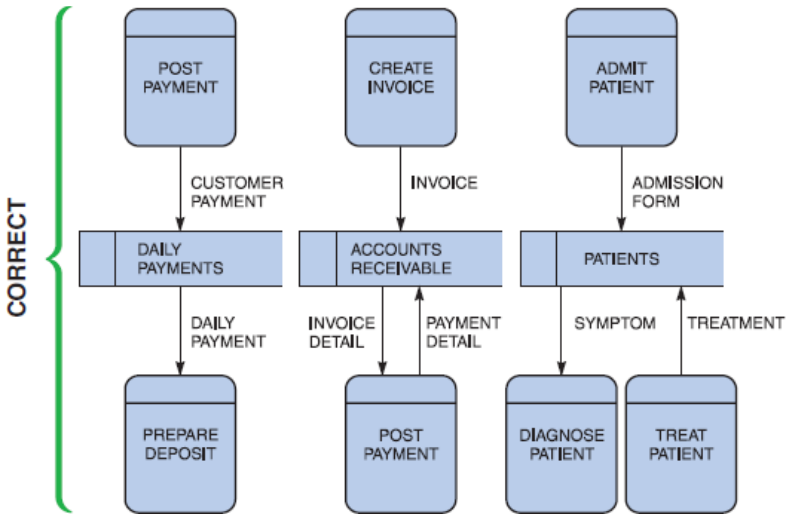


Figure 4: Examples of correct uses of data store symbols in a data flow diagram.

Figure 4 illustrates typical examples of data stores. In each case, the data store has at least one incoming and one outgoing data flow and is connected to a process symbol with a data flow.

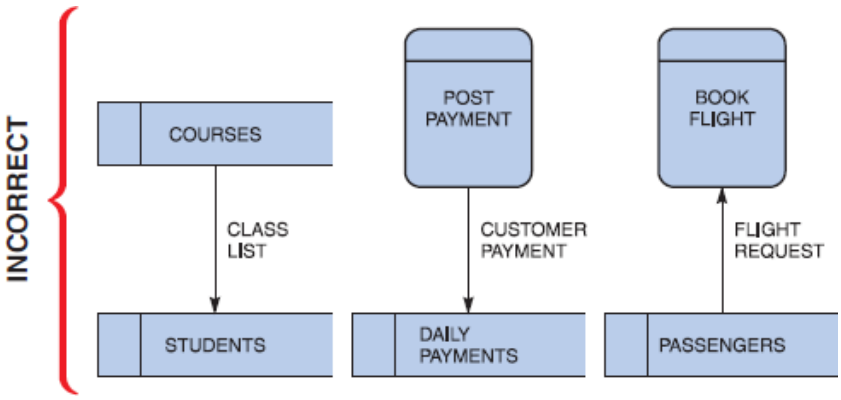


Figure 5: Examples of incorrect uses of data store symbols: Two data stores cannot be connected by a data flow without an intervening process, and each data store should have an outgoing and incoming data flow.

Violations of the rule that a data store must have at least one incoming and one outgoing data flow are shown in Figure 5. In the first example, two data stores are connected incorrectly because no process is between them. Also, COURSES has no incoming data flow and STUDENTS has no outgoing data flow. In the second and third examples, the data stores lack either an outgoing or incoming data flow.

Entity Symbol

- A DFD shows only external entities that provide data to the system or receive output from the system.
- DFD entities also are called terminators, because they are data origins or final destinations.

Figure 6: Examples of correct uses of external entities in a data flow diagram.

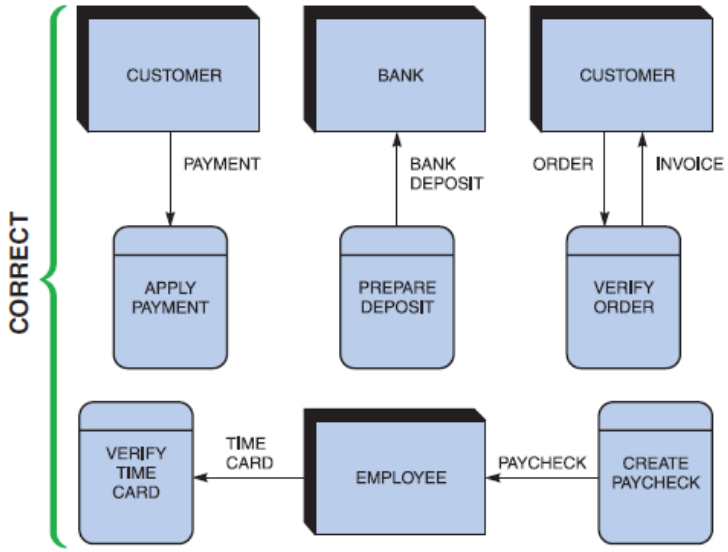


Figure 7: Examples of incorrect uses of external entities. An external entity must be connected by a data flow to a process, and not directly to a data store or to another external entity.

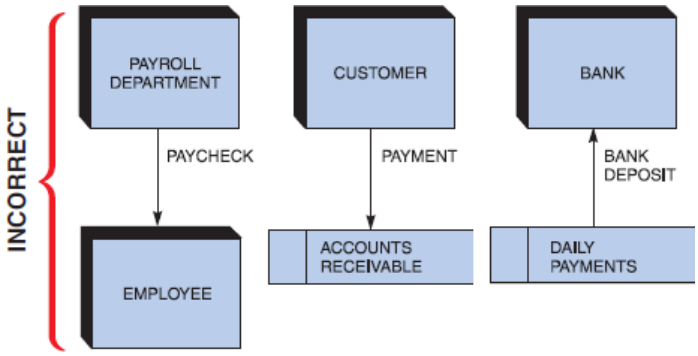
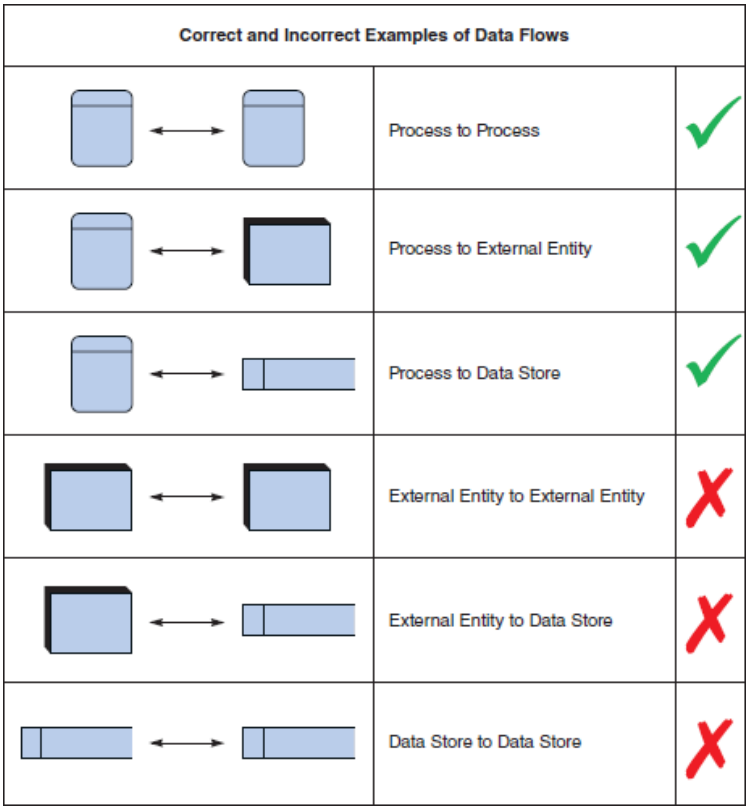


Figure 8 shows a summary of the rules for using DFD symbols.



Guidelines for Drawing DFDs

When you draw a context diagram and other DFDs, you should follow several guidelines:

- Draw the context diagram so it fits on one page.
- Use the name of the information system as the process name in the context diagram. For example, the process name in Figure 9 is GRADING SYSTEM. Notice that the process name is the same as the system name. For processes in lower-level DFDs, you would use a verb followed by a descriptive noun, such as ESTABLISH GRADEBOOK, ASSIGN FINAL GRADE, or PRODUCE GRADE REPORT.
- Use unique names within each set of symbols. For instance, the diagram in below figure shows only one entity named STUDENT and only one data flow named FINAL GRADE. Whenever you see the entity STUDENT on any other DFD in the grading system, you know that you are dealing with the same entity. Whenever the FINAL GRADE data flow appears, you know that you are dealing with the same data flow. The naming convention also applies to data stores.
- Do not cross lines. One way to achieve that goal is to restrict the number of symbols in any DFD. On lower-level diagrams with multiple processes, you should not have more than nine process symbols. Including more than nine symbols usually is a signal that your diagram is too complex and that you should reconsider your analysis.
- Provide a unique name and reference number for each process. Because it is the highest-level DFD, the context diagram contains process 0, which represents the entire information system, but does not show the internal workings. To describe the next level of detail inside process 0, you must create a DFD named diagram 0, which will reveal additional processes that must be named and numbered (See figure 10). As you continue to create lower-level DFDs, you assign unique names and reference numbers to all processes, until you complete the logical model.
- Obtain as much user input and feedback as possible. Your main objective is to ensure that the model is accurate, easy to understand, and meets the needs of its users.

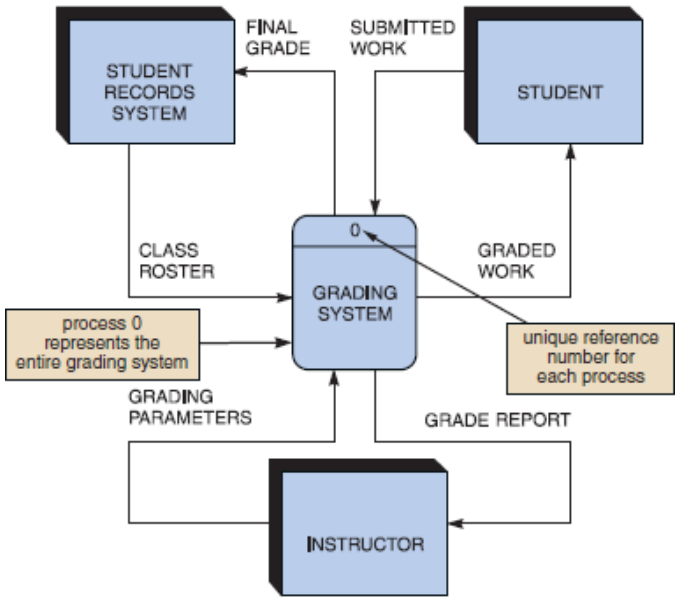


Figure 9: Context diagram DFD for a grading system.

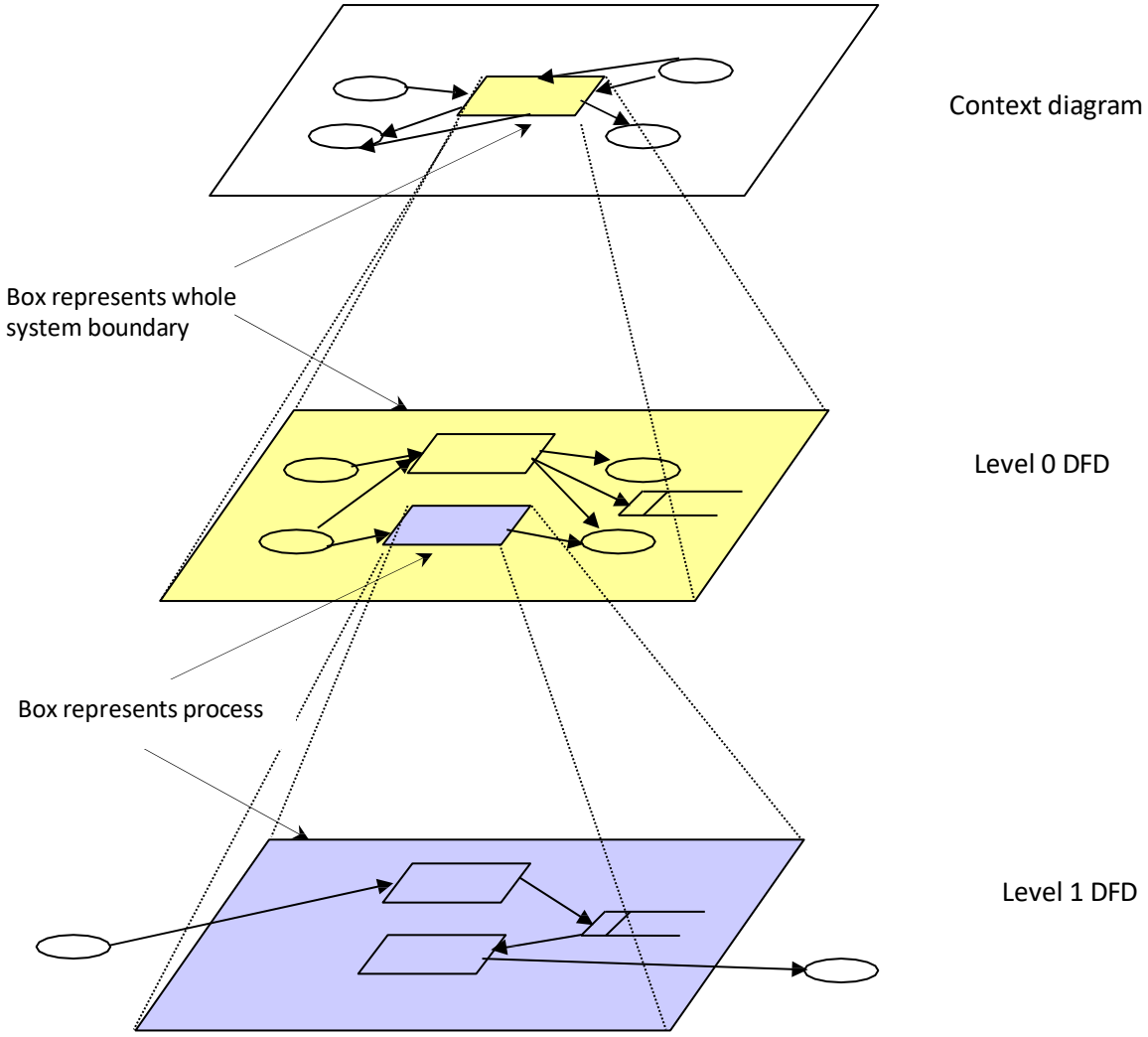


Figure 10: Different levels of DFD

Developing Data Flow Diagrams

Step 1: Draw a Context Diagram

A context diagram is a top-level view of an information system that shows the system's boundaries and scope. To draw a context diagram, you have to use a single process symbol, and you identify it as process 0.

Example: context diagram for an order system

The context diagram for an order system is shown in Figure 11. Notice that the ORDER SYSTEM process is at the center of the diagram and five entities surround the process. Three of the entities, SALES REP, BANK, and ACCOUNTING, have nine data flow.

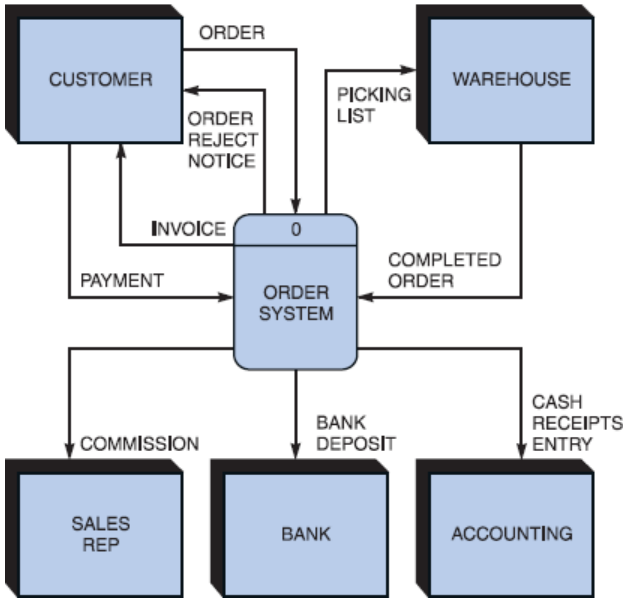


Figure 11: Context diagram DFD for an order system.

Step 2: Draw a Diagram 0 DFD

To show the detail inside the black box, you create DFD diagram 0. Diagram 0 zooms in on the system and shows major internal processes, data flows, and data stores. Diagram 0 also repeats the entities and data flows that appear in the context diagram. When you expand the context diagram into DFD diagram 0, you must retain all the connections that flow into and out of process 0.

Example: Diagram 0 DFD for an order system

Figure 12 shows the diagram 0 for an order system. Process 0 on the order system's context diagram is exploded to reveal three processes (FILL ORDER, CREATE INVOICE, and APPLY PAYMENT), one data store (ACCOUNTS RECEIVABLE), two additional data flows (INVOICE DETAIL and PAYMENT DETAIL), and one diverging data flow (INVOICE). The following walkthrough explains the DFD shown in Figure 12:

1. A CUSTOMER submits an ORDER. Depending on the processing logic, the FILL ORDER process either sends an ORDER REJECT NOTICE back to the customer or sends a PICKING LIST to the WAREHOUSE.
2. A COMPLETED ORDER from the WAREHOUSE is input to the CREATE INVOICE process, which outputs an INVOICE to both the CUSTOMER process and the ACCOUNTS RECEIVABLE data store.
3. A CUSTOMER makes a PAYMENT that is processed by APPLY PAYMENT. APPLY PAYMENT requires INVOICE DETAIL input from the ACCOUNTS RECEIVABLE data store along with the PAYMENT. APPLY PAYMENT also outputs PAYMENT DETAIL back to the ACCOUNTS RECEIVABLE data store and outputs COMMISSION to the SALES DEPT, BANK DEPOSIT to the BANK, and CASH RECEIPTS ENTRY to ACCOUNTING.

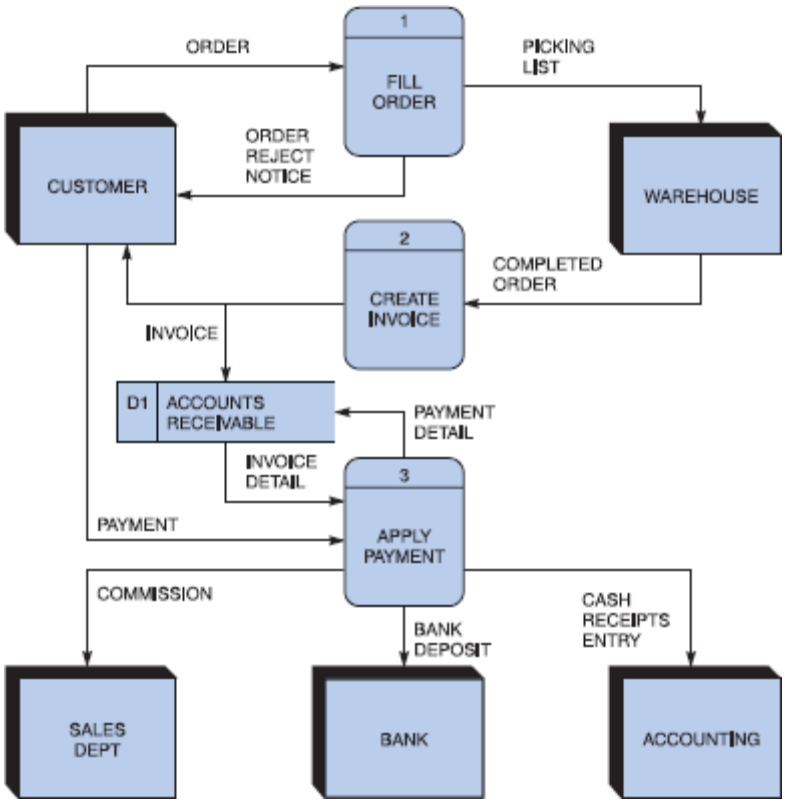


Figure 12: Diagram 0 DFD for the order system.

Step 3: Draw the Lower-Level Diagrams

This set of lower-level DFDs is based on the order system. To create lower-level diagrams, you must use leveling and balancing techniques. Leveling is the process of drawing a series of increasingly detailed diagrams, until all functional primitives are identified. Balancing maintains consistency among a set of DFDs by ensuring that input and output data flows align properly. Leveling and balancing are described in more detail in the following sections.

Leveling examples

Leveling uses a series of increasingly detailed DFDs to describe an information system. For example, a system might consist of dozens, or even hundreds, of separate processes. Using leveling, an analyst starts with an overall view, which is a context diagram with a single process symbol. Next, the analyst creates diagram 0, which shows more detail. The analyst continues to create lower-level DFDs which is the parent. When you explode the FILL ORDER process into diagram 1 DFD, however, you see that three processes (1.1, 1.2, and 1.3) interact with the two data stores, which now are shown.

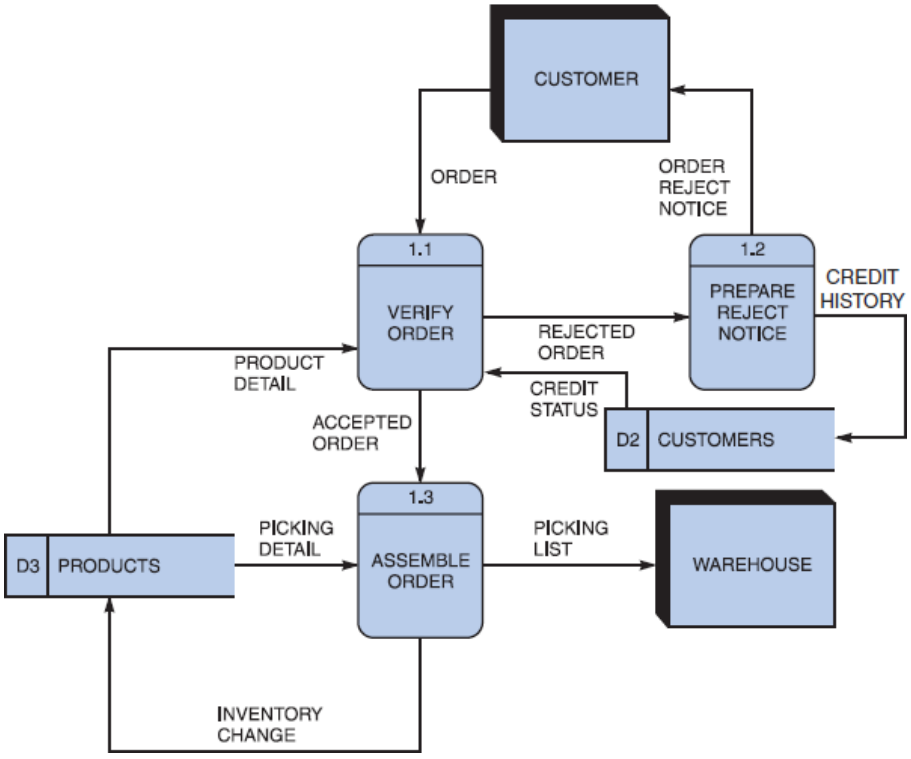


Figure 13: Diagram 1 DFD shows details of the FILL ORDER process in the order system.

Figures 12 and 13 provide an example of leveling. Figure 12 shows diagram 0 for an order system, with the FILL ORDER process labeled as process 1. Now consider Figure 5-17, which provides an exploded view of the FILL ORDER process. Notice that FILL ORDER (process 1) actually consists of three processes: VERIFY ORDER (process 1.1), PREPARE REJECT NOTICE (process 1.2), and ASSEMBLE ORDER (process 1.3).

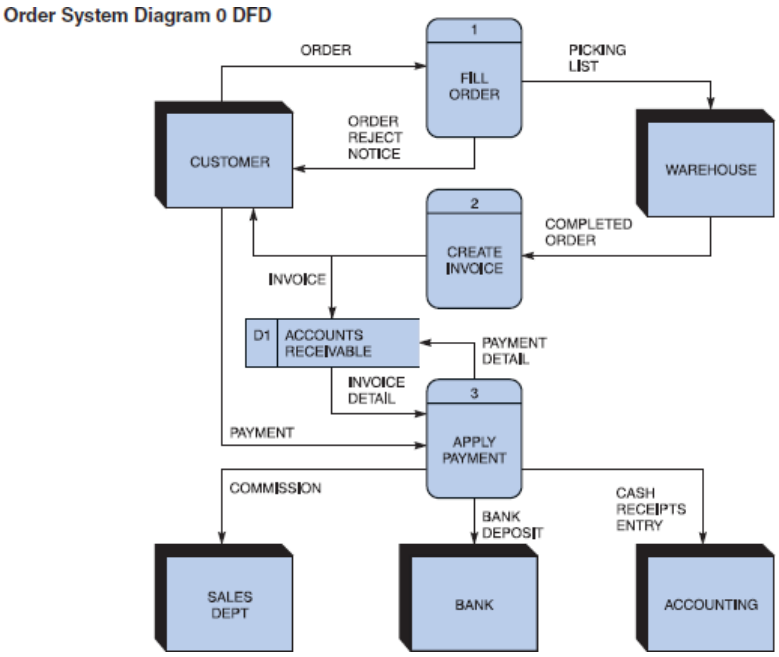
When you compare Figures 12 and 13, you will notice that Figure 13 (the exploded FILL ORDER process) shows two data stores (CUSTOMERS and PRODUCTS) that do not appear on Figure 5-16, which is the parent DFD. Why not? The answer is based on a simple rule: When drawing DFDs, you show a data store only when two or more processes use that data store. The CUSTOMERS and PRODUCTS data stores were internal to the FILL ORDER process, so the analyst did not show them on diagram 0, which is the parent. When you explode the FILL ORDER process into diagram 1 DFD, however, you see that three processes (1.1, 1.2, and 1.3) interact with the two data stores, which now are shown.

BALANCING EXAMPLES

Balancing ensures that the input and output data flows of the parent DFD are maintained on the child DFD. For example, Figure 14 shows two DFDs: The order system diagram 0 is shown at the top of the figure, and the exploded diagram 3 DFD is shown at the bottom.

The two DFDs are balanced, because the child diagram at the bottom has the same input and output flows as the parent process 3 shown at the top. To verify the balancing, notice that the parent process 3, APPLY PAYMENT, has one incoming data flow from an external entity, and three outgoing data flows to external entities. Now examine the child DFD, which is diagram 3. Now, ignore the internal data flows and count the data flows to and from external entities. You

will see that the three processes maintain the same one incoming and three outgoing data flows as the parent process.



Next

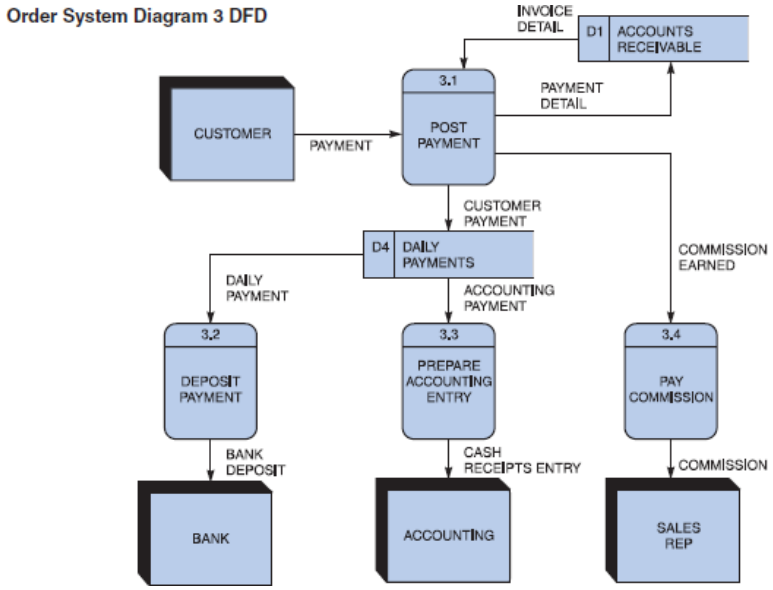


FIGURE 14: The order system diagram 0 is shown at the top of the figure, and exploded diagram 3 DFD (for the APPLY PAYMENT process) is shown at the bottom. The two DFDs are balanced, because the child diagram at the bottom has the same input and output flows as the parent process 3 shown at the top.

DFD EXAMPLE-2

Library Management System

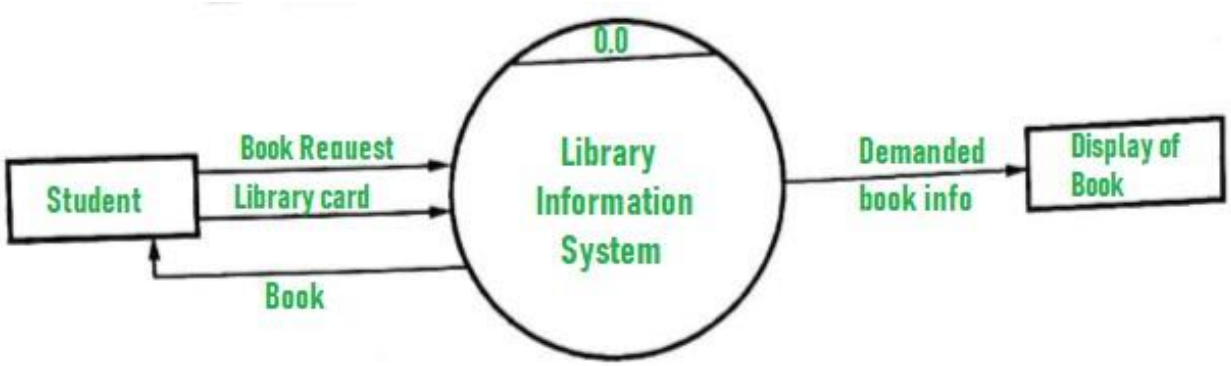


Figure: Context Diagram for Library Information System

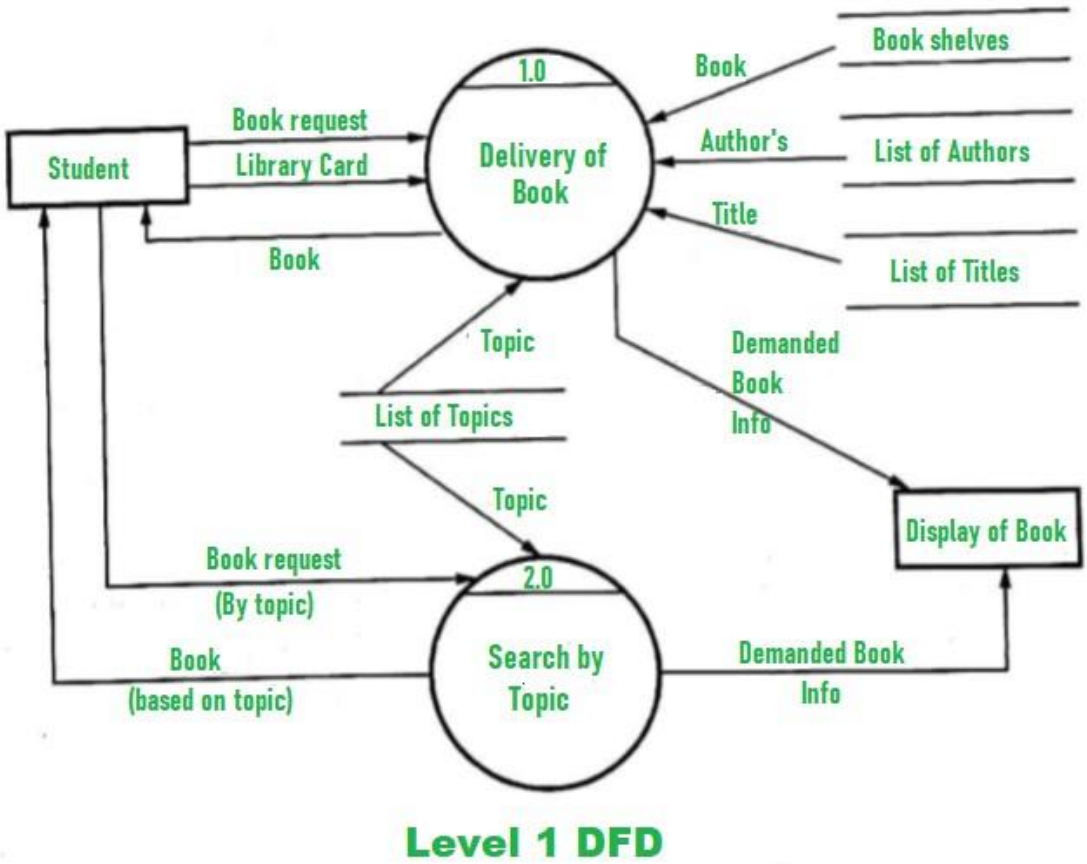


Figure: Level-1 DFD for Library Information System